

DIGITAL FORENSICS LABORATORY REPORT

Live Memory Forensics with Volatility 3

Acquisition and Plugin-Based Triage of a Windows Memory Image

DUMPIT · VOLATILITY 3 · WINDOWS MEMORY ANALYSIS

REPORT ID	DFCS · CCFA · VOLATILITY · 2026-07
PREPARED BY	Awais Ahmed
ACADEMIC PROGRAM	B.S. Digital Forensics and Cyber Security (DFCS)
CERTIFICATION TRACK	Certified Computer Forensics Analyst (CCFA)
SUBJECT MEDIA	Full physical memory image of the examiner's own HP laptop (LAPTOP-QRAGDIG9), ~ 5.9 GB
ACQUISITION TOOL	DumpIt (h4sh5/DumpIt-mirror)
ANALYSIS TOOL	Volatility 3 Framework 2.28.1 (Python 3.12)
REPORT DATE	July 23, 2026
CLASSIFICATION	Academic Coursework: Portfolio Release Copy

This document reproduces coursework performed against a live memory capture of the examiner's own personal system, in a controlled lab environment, for digital forensics education. All steps were independently executed by the author.

Report Details

Field	Detail
Examiner	Awais Ahmed
Role	Digital Forensics Student / Examiner-in-Training
Source Practical	"Volatility" memory-forensics worksheet
Subject Media	Physical memory image of the examiner's own system, captured for this exercise

Table of Contents

Report Details	2
Table of Contents	2
1. Executive Summary	3
2. Scope and Authorized Use	3
3. Background: Why Memory Forensics Matters.....	3
4. Examination Environment and Tools	3
5. Procedure and Results.....	4
5.1 Installing Volatility 3.....	4
5.2 Acquiring the Memory Image with DumpIt.....	4
5.3 Identifying Available Windows Plugins.....	5
5.4 windows.info: System Identification	5
5.5 windows.pslist: Running Process List.....	5
5.6 windows.netscan: Network Connections.....	6
5.7 windows.pstree: Process Hierarchy	6
5.8 windows.cmdline: Process Command-Line Arguments.....	7
5.9 windows.malware.malfind: Injected Code Detection	7
5.10 windows.filescan: Open File Handles	8
6. Narrowing an Investigation: Targeted Filtering.....	9
7. Findings Summary and Conclusion.....	9

1. Executive Summary

This report documents a live memory (RAM) forensics exercise performed as part of Digital Forensics and Cyber Security (DFCS) coursework aligned to Certified Computer Forensics Analyst (CCFA) competencies. A full physical memory image was captured from the examiner's own Windows 10 laptop using DumpIt, then analyzed with the Volatility 3 Framework to demonstrate the standard triage workflow an examiner would follow against any Windows memory capture.

Seven Volatility plugins were run against the image in sequence, windows.info, windows.pslist, windows.netscan, windows.pstree, windows.cmdline, windows.malware.malfind, and windows.filescan, each targeting a different category of volatile evidence: system identification, running processes, active network connections, process ancestry, process launch arguments, injected-code detection, and open file handles.

No malicious activity was identified. windows.malware.malfind flagged three processes with executable-writable memory regions, all of which were assessed as expected false positives tied to legitimate OEM and antivirus software rather than injected code (Section 5.6). The report closes with the command syntax for narrowing a Volatility search around a specific suspicious process, which would be the next step if this had been a live incident-response engagement rather than a clean baseline system.

2. Scope and Authorized Use

All actions in this report were performed against a memory image of the examiner's own personal system, captured specifically for this coursework exercise, in a controlled lab environment. Capturing and analyzing full system memory is a standard, lawful step in an authorized digital forensics or incident-response engagement; performing the same capture against a system one does not own or have written authorization to examine would not be lawful. This report should be read strictly as a record of coursework against the examiner's own machine, not as guidance for use against systems without proper authorization.

3. Background: Why Memory Forensics Matters

Traditional ("dead-box") forensics, as in the companion Windows Registry examination of this portfolio, recovers evidence from a disk image after the system has been shut down. Memory forensics instead captures the contents of RAM while a system is running, and analyzes that snapshot afterward. This matters because a large amount of forensically valuable information exists only in memory and is lost the moment a system powers off or a process exits:

- Running processes and their exact command-line arguments, including ones that never wrote a trace to disk.
- Active and recently-closed network connections, tied to the specific process that owned them.
- Decrypted data, plaintext credentials, or encryption keys that exist in memory only while a program is actively using them.
- Malicious code injected directly into a legitimate process's memory space, which may never exist as a file on disk at all.

Volatility 3 is the current generation of the Volatility Framework, an open-source tool purpose-built to parse these raw memory images and reconstruct operating-system structures (the process list, network tables, loaded modules, and so on) directly from the raw bytes of a memory dump, without needing the original live system.

4. Examination Environment and Tools

- Python 3.12, required runtime for Volatility 3.
- Volatility 3 Framework 2.28.1, installed in editable mode from source with the full optional dependency set (yara-python, capstone, pycryptodome, leechcorepyc, pefile), enabling malware-scanning and disassembly features used later in this report.

Volatility 3 source: <https://github.com/volatilityfoundation/volatility3>

- DumpIt (h4sh5/DumpIt-mirror), a lightweight Windows memory-acquisition utility used to capture the full physical memory image analyzed in this report.

DumpIt-mirror source: <https://github.com/h4sh5/DumpIt-mirror>

5. Procedure and Results

5.1 Installing Volatility 3

Volatility 3 was installed in editable mode with its full optional dependency set, which pulls in pefile, yara-python, capstone, pycryptodome, and leechcorepyc, the packages that specifically enable PE parsing, YARA pattern scanning, disassembly, and low-level memory-layer support used by later plugins in this report.

```
pip install --user -e ".[full]"
```

```
C:\Users\Awais\Desktop\CCFA\3. Tools\volatility3-develop>pip install --user -e ".[full]"
Obtaining file:///C:/Users/Awais/Desktop/CCFA/3.%20Tools/volatility3-develop
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Collecting pefile>=2024.8.26 (from volatility3==2.28.1)
  Downloading pefile-2024.8.26-py3-none-any.whl.metadata (1.4 kB)
Collecting yara-python<5,>=4.5.1 (from volatility3==2.28.1)
  Downloading yara_python-4.5.4-cp312-cp312-win_amd64.whl.metadata (3.0 kB)
Collecting capstone<6,>=5.0.3 (from volatility3==2.28.1)
  Downloading capstone-5.0.9-py3-none-win_amd64.whl.metadata (3.4 kB)
Collecting pycryptodome<4,>=3.21.0 (from volatility3==2.28.1)
  Downloading pycryptodome-3.23.0-cp37-abi3-win_amd64.whl.metadata (3.5 kB)
Collecting leechcorepyc<3,>=2.19.2 (from volatility3==2.28.1)
  Downloading leechcorepyc-2.22.12-cp36-abi3-win_amd64.whl.metadata (536 bytes)
Requirement already satisfied: pillow<11.0.0,>=10.0.0 in c:\users\awais\appdata\local\programs\python\python312\lib\
-packages (from volatility3==2.28.1) (10.3.0)
Downloading capstone-5.0.9-py3-none-win_amd64.whl (1.3 MB)
----- 1.3/1.3 MB 250.8 kB/s eta 0:00:00
Downloading leechcorepyc-2.22.12-cp36-abi3-win_amd64.whl (494 kB)
Downloading pefile-2024.8.26-py3-none-any.whl (74 kB)
Downloading pycryptodome-3.23.0-cp37-abi3-win_amd64.whl (1.8 MB)
----- 1.8/1.8 MB 83.5 kB/s eta 0:00:00
Downloading yara_python-4.5.4-cp312-cp312-win_amd64.whl (1.8 MB)
----- 1.8/1.8 MB 132.0 kB/s eta 0:00:00
```

Figure 1. Installing Volatility 3 and its full dependency set via pip.

5.2 Acquiring the Memory Image with DumpIt

DumpIt was run directly on the target system and, when prompted, confirmed with 'y' to begin capture. It wrote the full physical memory contents to a timestamped .dmp file, alongside a small companion .json metadata file.

Name	Date modified	Type	Size
DumpIt	13/10/2021 5:37 pm	Application	519 KB
LAPTOP-QRAGDJG9-20260722-053441.dmp	22/07/2026 10:37 am	DMP File	6,168,164 KB
LAPTOP-QRAGDJG9-20260722-053441.json	22/07/2026 10:37 am	JSON File	2 KB

Figure 2. DumpIt's output folder: the captured memory image (LAPTOP-QRAGDIG9-20260722-053441.dmp, ~ 5.9 GB) and its .json metadata file.

Finding: A complete physical memory image ($\approx 6,168,164$ KB) was captured successfully and used as the input file for every Volatility command that follows.

5.3 Identifying Available Windows Plugins

Volatility 3 organizes its plugins by operating system (windows.*, linux.*, mac.*). Running the tool with just the bare category name windows returns a disambiguation error listing every plugin available under that category, this is what produced the full plugin listing below, which serves as a reference for the operating-system-level artifacts Volatility 3 can extract from a Windows memory image.

```
usage: vol.py [-h] [-c CONFIG] [--parallelism [{processes,threads,off}]] [-e EXTEND] [-p PLUGIN_DIRS] [-s SYMBOL_DIRS] [-v] [-l LOG] [-o OUTPUT_DIR] [-q] [-f FILE]
             [--write-config] [--save-config SAVE_CONFIG] [--clear-cache] [--cache-path CACHE_PATH] [--offline] [-u URL] [--filters FILTERS]
             [--hide-columns [HIDE_COLUMNS ...]] [-r RENDERER] [--single-location SINGLE_LOCATION] [--stackers [STACKERS ...]]
             [--single-swap-locations [SINGLE_SWAP_LOCATIONS ...]]
             PLUGIN ...

vol.py: error: argument PLUGIN: plugin windows matches multiple plugins (windows.amcache.Amcache, windows.bigpools.BigPools, windows.cachedump.Cachedump, windows.callba
cks.Callbacks, windows.cmdline.Cmdline, windows.cmdscan.CmdScan, windows.consoles.Consoles, windows.crashinfo.CrashInfo, windows.debugregisters.DebugRegisters, windows.
deskscan.DeskScan, windows.desktops.Desktops, windows.devicetree.DeviceTree, windows.direct_system_calls.DirectSystemCalls, windows.dlllist.Dlllist, windows.driverirp.D
riverIrp, windows.drivermodule.DriverModule, windows.driverscan.DriverScan, windows.dumpfiles.DumpFiles, windows.envvars.Envvars, windows.etwpatch.EtwPatch, windows.files
can.FileScan, windows.getservicesids.GetServicesIDs, windows.getsysids.GetSysIDs, windows.handles.Handles, windows.hashdump.Hashdump, windows.hollowprocesses.HollowProcesse
s, windows.ist.IAT, windows.indirect_system_calls.IndirectSystemCalls, windows.info.Info, windows.joblinks.JobLinks, windows.kpers.KPCRS, windows.ldrmodules.LdrModules,
windows.lsadump.Lsadump, windows.malfind.Malfind, windows.malware.direct_system_calls.DirectSystemCalls, windows.malware.drivermodule.DriverModule, windows.malware.hol
lowprocesses.HollowProcesses, windows.malware.indirect_system_calls.IndirectSystemCalls, windows.malware.ldrmodules.LdrModules, windows.malware.malfind.Malfind, windows
.malware.pebmasquerade.PebMasquerade, windows.malware.processghosting.ProcessGhosting, windows.malware.psxview.PsxView, windows.malware.skeleton_key_check.SkeletonKey
Check, windows.malware.suspicious_threads.SuspiciousThreads, windows.malware.svcdiff.SvcDiff, windows.malware.unhooked_system_calls.UnhookedSystemCalls, windows.mbrscan
.MbrScan, windows.memmap.Memmap, windows.mftscan.MftScan, windows.mftscan.ResidentData, windows.modscan.ModScan, windows.modules.Modules, windows.m
utantscan.Mutantscan, windows.netscan.NetScan, windows.netstat.NetStat, windows.orphan_kernel_threads.Threads, windows.pe_symbols.PESymbols, windows.pedump.PEDump, wind
ows.poolscanner.PoolScanner, windows.privileges.Privs, windows.processghosting.ProcessGhosting, windows.pslist.PsList, windows.psscan.PsScan, windows.psree.PsTree, win
dows.psxview.PsxView, windows.registry.amcache.Amcache, windows.registry.cachedump.Cachedump, windows.registry.certificates.Certificates, windows.registry.getcellroutin
e.GetCellRoutine, windows.registry.hashdump.Hashdump, windows.registry.hivelist.HiveList, windows.registry.hivescan.HiveScan, windows.registry.lsadump.Lsadump, windows.
registry.printkey.PrintKey, windows.registry.scheduled_tasks.ScheduledTasks, windows.registry.userassist.UserAssist, windows.scheduled_tasks.ScheduledTasks, windows.ses
sions.Sessions, windows.shimcachemem.ShimCacheMem, windows.skeleton_key_check.SkeletonKeyCheck, windows.ssd.SSD, windows.statistics.Statistics, windows.strings.Strin
gs, windows.suspended_threads.SuspendedThreads, windows.suspicious_threads.SuspiciousThreads, windows.svcdiff.SvcDiff, windows.svclist.SvcList, windows.svcscan.SvcScan
, windows.symblinkscan.SymLinkScan, windows.thrdscan.ThrdScan, windows.threads.Threads, windows.timers.Timers, windows.truecrypt.Passphrase, windows.unhooked_system_call
s.unhooked_system_calls, windows.unloadedmodules.UnloadedModules, windows.vadinfo.VadInfo, windows.vadregexscan.VadRegExScan, windows.vadwalk.VadWalk, windows.vadvarasc
an.VadVaraScan, windows.verinfo.VerInfo, windows.virtmap.VirtMap, windows.windows.Windows, windows.windowstations.WindowStations)
```

Figure 3. Volatility 3's plugin disambiguation error, listing all available windows.* plugins (e.g. windows.pslist, windows.netscan, windows.malware.malfind, windows.registry.hashdump) as a category reference.

Of this full plugin catalog, the seven plugins used for this exercise are detailed in Sections 5.4–5.10 below, each with its purpose and findings.

5.4 windows.info: System Identification

Purpose: Confirms which build of Windows produced the memory image, the kernel base address, and the symbol table Volatility resolved against, establishing that the correct OS profile was used before trusting any later plugin's output.

```
python vol.py -f "...\\LAPTOP-QRAGDIG9-20260722-053441.dmp" windows.info
```

```
C:\Users\Awais\Desktop\CCFA\3. Tools\volatility3-develop>python vol.py -f "C:\Users\Awais\Desktop\CCFA\3. Tools\DumpIt-mirror-main\LAPTOP-QRAGDIG9-20260722-053441.dmp"
windows.info
Volatility 3 Framework 2.28.1
Progress: 89.0% Updating caches for 110 files...
```

Figure 4. windows.info command invocation.

```
Volatility 3 Framework 2.28.1
Progress: 100.0% PDB scanning finished
Variable Value
Kernel Base 0xf80329600000
DIB 0x1ae000
Symbols file:///C:/Users/Awais/Desktop/CCFA/3.%20Tools/volatility3-develop/volatility3/symbols/windows/ntkrnlmp.pdb/F57E740B088E5956E8AF072F1CC58EB-1.json.xz
Is64Bit True
IsPAE False
layer_name 0 WindowsIntel132e
memory_layer 1 WindowsCrashDump64layer
base_layer 2 FileLayer
KdVersionBlock 0xf8032a20f400
Major/Minor 15.19041
MachineType 34404
KdNumberProcessors 2
SystemTime 2026-07-22 05:36:30+00:00
NtSystemRoot C:\WINDOWS
NtProductType NtProductWinNT
NtMajorVersion 10
NtMinorVersion 0
PE MajorOperatingSystemVersion 10
PE MinorOperatingSystemVersion 0
PE Machine 34404
PE TimeDateStamp Sat Feb 2 23:04:03 1985
```

Figure 5. windows.info output: kernel base, symbol file, and system version details.

Finding: Windows 10 (NtMajorVersion 10), 2 logical processors, 64-bit, SystemTime 2026-07-22 05:36:30 UTC, confirming the image is a valid, correctly-profiled Windows 10 memory capture matching the capture time in Section 5.2.

5.5 windows.pslist: Running Process List

Purpose: Enumerates every active process at the moment of capture by walking the kernel's process list, giving PID, parent PID, thread/handle counts, and creation time for each, the starting point for almost any memory investigation.

```
C:\Users\Awais\Desktop\CCFA\3. Tools\volatility3-develop>python vol.py -f "C:\Users\Awais\Desktop\CCFA\3. Tools\DumpIt-mirror-main\LAPTOP-QRAGDJG9-20260722-053441.dmp"
Windows.pslist
Volatility 3 Framework 2.28.1
Progress: 100.00
PDB scanning finished
Offset(V) Threads Handles SessionId Wow64 CreateTime ExitTime File output
PID PPID ImageFileName
4 0 System 0xb8d170c0140 191 - N/A False 2026-07-15 12:02:22.000000 UTC N/A Disabled
80 4 Registry 0xb8d1713b000 2 - N/A False 2026-07-15 12:02:20.000000 UTC N/A Disabled
404 4 smss.exe 0xb8d1a384000 2 - N/A False 2026-07-15 12:02:22.000000 UTC N/A Disabled
712 516 csrss.exe 0xb8d2042e140 10 - 0 False 2026-07-15 12:02:27.000000 UTC N/A Disabled
868 516 wininit.exe 0xb8d23ae92c0 1 - 0 False 2026-07-15 12:02:28.000000 UTC N/A Disabled
1004 868 services.exe 0xb8d1a122300 6 - 0 False 2026-07-15 12:02:28.000000 UTC N/A Disabled
1016 868 lsass.exe 0xb8d2436d000 8 - 0 False 2026-07-15 12:02:28.000000 UTC N/A Disabled
508 1004 svchost.exe 0xb8d207202c0 13 - 0 False 2026-07-15 12:02:28.000000 UTC N/A Disabled
```

Figure 6. windows.pslist output: System, Registry, smss.exe, csrss.exe, wininit.exe, services.exe, lsass.exe, and further processes with their PIDs and creation times.

Finding: A normal, expected core Windows process set was present (System, Registry, smss.exe, csrss.exe, wininit.exe, services.exe, lsass.exe), each with a plausible parent PID and no unexpected process names, no findings of concern at this stage.

5.6 windows.netscan: Network Connections

Purpose: Scans memory for network-connection structures (TCP/UDP endpoints), independent of the live pslist walk, recovering both active and some recently-closed connections along with the owning process, which is useful for identifying command-and-control traffic, data exfiltration, or unexpected listening services.

```
C:\Users\Awais\Desktop\CCFA\3. Tools\volatility3-develop>python vol.py -f "C:\Users\Awais\Desktop\CCFA\3. Tools\DumpIt-mirror-main\LAPTOP-QRAGDJG9-20260722-053441.dmp"
Windows.netscan
Volatility 3 Framework 2.28.1
Progress: 100.00
PDB scanning finished
Offset LocalPort ForeignAddr ForeignPort State PID Owner Created
TCPv4 172.60.34.189 58468 57.155.141.122 443 ESTABLISHED 4808 MsMpEng.exe 2026-07-22 05:36:29.000000 UTC
TCPv4 0.0.0.0 5357 0.0.0.0 LISTENING 4 System 2026-07-17 04:06:32.000000 UTC
TCPv6 :: 5357 :: 0 LISTENING 4 System 2026-07-17 04:06:32.000000 UTC
TCPv4 172.60.34.189 139 0.0.0.0 LISTENING 4 System 2026-07-22 04:49:45.000000 UTC
TCPv4 172.60.34.189 58354 204.79.197.222 443 CLOSED 8264 SearchApp.exe 2026-07-22 05:22:17.000000 UTC
TCPv4 172.60.34.189 58471 20.207.73.85 443 ESTABLISHED 12228 brave.exe 2026-07-22 05:36:57.000000 UTC
TCPv4 172.60.34.189 58425 185.199.108.133 443 ESTABLISHED 12228 brave.exe 2026-07-22 05:31:13.000000 UTC
TCPv4 0.0.0.0 49664 0.0.0.0 LISTENING 1016 lsass.exe 2026-07-15 12:02:28.000000 UTC
TCPv6 :: 49664 :: 0 LISTENING 1016 lsass.exe 2026-07-15 12:02:28.000000 UTC
TCPv4 0.0.0.0 135 0.0.0.0 LISTENING 1128 svchost.exe 2026-07-15 12:02:28.000000 UTC
TCPv4 0.0.0.0 49665 0.0.0.0 LISTENING 868 wininit.exe 2026-07-15 12:02:28.000000 UTC
TCPv6 :: 49665 :: 0 LISTENING 868 wininit.exe 2026-07-15 12:02:28.000000 UTC
TCPv4 0.0.0.0 49666 0.0.0.0 LISTENING 1728 svchost.exe 2026-07-15 12:02:29.000000 UTC
TCPv4 0.0.0.0 49664 0.0.0.0 LISTENING 1016 lsass.exe 2026-07-15 12:02:28.000000 UTC
TCPv4 0.0.0.0 135 0.0.0.0 LISTENING 1128 svchost.exe 2026-07-15 12:02:28.000000 UTC
TCPv6 :: 135 :: 0 LISTENING 1128 svchost.exe 2026-07-15 12:02:28.000000 UTC
TCPv4 0.0.0.0 49665 0.0.0.0 LISTENING 868 wininit.exe 2026-07-15 12:02:28.000000 UTC
TCPv4 0.0.0.0 49666 0.0.0.0 LISTENING 1728 svchost.exe 2026-07-15 12:02:29.000000 UTC
TCPv6 :: 49666 :: 0 LISTENING 1728 svchost.exe 2026-07-15 12:02:29.000000 UTC
TCPv4 172.60.34.189 58356 150.171.22.254 443 CLOSED 8264 SearchApp.exe 2026-07-22 05:22:54.000000 UTC
TCPv4 172.60.34.189 58206 23.103.242.200 443 ESTABLISHED 7352 WINWORD.EXE 2026-07-22 04:56:45.000000 UTC
TCPv4 172.60.34.189 58358 204.79.197.203 80 CLOSED 8264 SearchApp.exe 2026-07-22 05:22:56.000000 UTC
TCPv4 172.60.34.189 58417 23.185.0.2 443 ESTABLISHED 12228 brave.exe 2026-07-22 05:30:34.000000 UTC
TCPv4 172.60.34.189 58461 3.173.21.63 443 ESTABLISHED 12228 brave.exe 2026-07-22 05:34:54.000000 UTC
TCPv4 172.60.34.189 58462 43.109.76.178 443 CLOSE_WAIT 12228 brave.exe 2026-07-22 05:34:56.000000 UTC
TCPv4 172.60.34.189 58352 150.171.28.254 443 CLOSED 8264 SearchApp.exe 2026-07-22 05:22:10.000000 UTC
TCPv4 172.60.34.189 58074 4.213.25.241 443 ESTABLISHED 4836 svchost.exe 2026-07-22 04:49:47.000000 UTC
```

Figure 7. windows.netscan output: TCP/IPv4 and IPv6 endpoints with local/foreign address, port, state, and owning process.

Finding: All identified connections resolved to expected, legitimate processes: MsMpEng.exe (Windows Defender, ESTABLISHED to a Microsoft telemetry/definition-update endpoint), brave.exe (multiple ESTABLISHED HTTPS connections, consistent with normal browsing), SearchApp.exe (Windows Search, CLOSED connections to Microsoft/Bing endpoints), and svchost.exe/lsass.exe/wininit.exe listening on standard Windows service ports (135, 139, 49664–49666). No connections to unrecognized or suspicious foreign addresses were observed.

5.7 windows.pstree: Process Hierarchy

Purpose: Presents the same process list as pslist but organized by parent-child relationship (indentation shows which process launched which), which is useful for spotting a process running from an unexpected parent, a classic sign of process injection or masquerading.


```

C:\Users\Awais\Desktop\CCFA\3. Tools\volatility3-develop>python vol.py -f "C:\Users\Awais\Desktop\CCFA\3. Tools\DumpIt-mirror-main\LAPTOP-QRAGD7G9-20260722-053441.dmp"
windows.pstree
Volatility 3 Framework 2.28.1
Progress: 100.00 PDB scanning finished
PID PPID ImageFileName OffSet(V) Threads Handles SessionId Wow64 CreateTime ExitTime Audit Cmd Path
4 0 System 0xb8d170b140 101 - N/A False 2026-07-15 12:02:22.000000 UTC N/A - - -
80 4 Registry 0xb8d1713b000 4 - N/A False 2026-07-15 12:02:20.000000 UTC N/A - - -
2740 4 MemCompression 0xb8d1714f040 758 - N/A False 2026-07-15 12:02:30.000000 UTC N/A - - -
404 4 smss.exe 0xb8d1a384000 2 - N/A False 2026-07-15 12:02:22.000000 UTC N/A - - -
\Device\HarddiskVolume3\Windows\System32\smss.exe
516 csrss.exe 0xb8d2042e140 10 - 0 False 2026-07-15 12:02:27.000000 UTC N/A - - -
\Device\HarddiskVolume3\Windows\System32\csrss.exe
\%SystemRoot%\system32\csrss.exe ObjectDirectory=\Windows SharedSection=1024,20480,768 Windows=On SubSystemType=Windows ServerDll=basesrv,1 ServerDll=winuser
ServerDll=wininit,2 ServerDll=ssxsr,4 ProfileControl=Off MaxRequestThreads=16 C:\WINDOWS\system32\csrss.exe
516 wininit.exe 0xb8d23ae92c0 1 - 0 False 2026-07-15 12:02:28.000000 UTC N/A - - -
\Device\HarddiskVolume3\Windows\System32\wininit.exe
1016 868 lsass.exe 0xb8d2436d200 8 - 0 False 2026-07-15 12:02:28.000000 UTC N/A - - -
\Device\HarddiskVolume3\Windows\System32\lsass.exe
C:\WINDOWS\system32\lsass.exe C:\WINDOWS\system32\lsass.exe
1004 868 services.exe 0xb8d1a122300 6 - 0 False 2026-07-15 12:02:28.000000 UTC N/A - - -
\Device\HarddiskVolume3\Windows\System32\services.exe
C:\WINDOWS\system32\services.exe C:\WINDOWS\system32\services.exe
4612 1004 svchost.exe 0xb8d25507800 1 - 0 False 2026-07-15 12:02:36.000000 UTC N/A - - -
\Device\HarddiskVolume3\Windows\System32\svchost.exe
C:\WINDOWS\system32\svchost.exe -k LocalService -p -s SstpSvc C:\WINDOWS\system32\svchost.exe

```

Figure 8. windows.pstree output showing System → smss.exe / Registry / MemCompression, and csrss.exe / wininit.exe → lsass.exe, services.exe → svchost.exe, with full paths and command lines.

Finding: Every process observed had a parent consistent with the normal Windows Vista/10 boot sequence (System spawning smss.exe and Registry; wininit.exe spawning services.exe and lsass.exe; services.exe spawning svchost.exe instances), no orphaned or unexpectedly-parented processes were found.

5.8 windows.cmdline: Process Command-Line Arguments

Purpose: Recovers the exact command line each process was launched with, including service-hosting arguments passed to generic executables like svchost.exe, which is critical for distinguishing a legitimate system service from a malicious process disguising itself under a common process name.

```

C:\Users\Awais\Desktop\CCFA\3. Tools\volatility3-develop>python vol.py -f "C:\Users\Awais\Desktop\CCFA\3. Tools\DumpIt-mirror-main\LAPTOP-QRAGD7G9-20260722-053441.dmp"
windows.cmdline
Volatility 3 Framework 2.28.1
Progress: 100.00 PDB scanning finished
PID Process Args
4 System -
80 Registry -
404 smss.exe \SystemRoot\System32\smss.exe
712 csrss.exe %SystemRoot%\system32\csrss.exe ObjectDirectory=\Windows SharedSection=1024,20480,768 Windows=On SubSystemType=Windows ServerDll=basesrv,1 Serve
rDll=wininit,2 ServerDll=ssxsr,4 ProfileControl=Off MaxRequestThreads=16
516 wininit.exe C:\WINDOWS\system32\services.exe
1004 services.exe C:\WINDOWS\system32\services.exe
1016 lsass.exe C:\WINDOWS\system32\lsass.exe
500 svchost.exe C:\WINDOWS\system32\svchost.exe -k DcomLaunch -p
1028 fontdrvhost.exe -
1128 svchost.exe C:\WINDOWS\system32\svchost.exe -k RPCSS -p
1180 svchost.exe C:\WINDOWS\system32\svchost.exe -k DcomLaunch -p -s LSM
1268 svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalServiceNetworkRestricted -s BThAgService
1284 svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalService -p -s BThAvctpsvc
1292 svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalService -p -s bthserv
1380 svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalSystemNetworkRestricted -p -s NcbService
1492 svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalServiceNetworkRestricted -p -s TimeBrokerSvc
1588 svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalServiceNetwork -p
1628 svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalSystemNetworkRestricted -p -s DisplayEnhancementService
1656 svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalSystemNetworkRestricted -p -s hidserv
1728 svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalServiceNetworkRestricted -p -s Eventlog
1872 svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalSystemNetworkRestricted -p -s TabletInputService

```

Figure 9. windows.cmdline output listing each process's full command line, including svchost.exe instances with their -k service-group arguments (e.g. -k LocalService -p -s BthAvctpsvc).

Finding: Every svchost.exe instance carried a plausible, named -k service-group argument (DcomLaunch, RPCSS, LocalServiceNetworkRestricted, EventLog, and others) matching known legitimate Windows service groups, none were found running with a blank, malformed, or suspicious argument, which would be the classic red flag for a process masquerading as svchost.exe.

5.9 windows.malware.malfind: Injected Code Detection

Purpose: Scans each process's memory for regions that are both writable and executable (PAGE_EXECUTE_READWRITE) with no backing file on disk, which is a strong indicator of injected shellcode or a reflectively-loaded malicious module, since legitimate code is normally either executable-only (loaded from a file) or writable-only (ordinary data), rarely both at once.

```
C:\Users\Awais\Desktop\CCFA\3. Tools\volatility3-devel\python vol.py -f "C:\Users\Awais\Desktop\CCFA\3. Tools\DumpIt-mirror-main\LAPTOP-QRAGD0G9-20260722-053441.dmp" --windows.malware.malfind
```

```
Volatility 3 Framework 2.28.1
Progress: 100.0% PDB scanning finished
PID Process Start VPN End VPN Tag Protection CommitChange PrivateMemory File output Notes Hexdump Disasm
```

```
2980 TouchpointAnal 0x299e97a0000 0x299e97affff VadS PAGE_EXECUTE_READWRITE 6 1 Disabled N/A
00 00 00 00 00 00 00 20 21 56 ae ac 76 00 01 ..... IV.V..
ee ff ee ff 02 00 00 18 00 6e ec 99 02 00 .....n.....
20 01 7a e9 99 02 00 00 00 00 7a e9 99 02 00 ..Z.....Z.....
00 00 7a e9 99 02 00 0f 00 00 00 00 00 00 ..z.....
0x299e97a0000: add byte ptr [rax], al
0x299e97a0002: add byte ptr [rax], al
0x299e97a0004: add byte ptr [rax], al
0x299e97a0006: add byte ptr [rax], al
0x299e97a0008: and byte ptr [rcx], ah
0x299e97a000a: push rsi
0x299e97a000b: scasd al, byte ptr [rdi]
0x299e97a000c: lodsb al, byte ptr [rsi]
0x299e97a000d: jbe 0x299e97a000f
0x299e97a000f: add esi, ebp
4420 HPCommRecovery 0x1b5bc010000 0x1b5bc01ffff VadS PAGE_EXECUTE_READWRITE 2 1 Disabled N/A
00 00 00 00 00 00 00 89 94 da cd 5f 28 00 01 .....(.....
ee ff ee ff 02 00 00 20 01 01 bc b5 01 00 00 .....
20 01 01 bc b5 01 00 00 00 00 01 bc b5 01 00 .....
00 00 01 bc b5 01 00 0f 00 00 00 00 00 00 .....
0x1b5bc010000: add byte ptr [rax], al
0x1b5bc010002: add byte ptr [rax], al
0x1b5bc010004: add byte ptr [rax], al
0x1b5bc010006: add byte ptr [rax], al
0x1b5bc010008: mov dword ptr [rdx + rbx*8 + 0x285fcd], edx
0x1b5bc01000f: add esi, ebp
4808 NtSpEng.exe 0x231262c0000 0x231262ccfff VadS PAGE_EXECUTE_READWRITE 269 1 Disabled N/A
=====
=====
```

Figure 10. windows.malware.malfind output flagging three processes with PAGE_EXECUTE_READWRITE regions: TouchpointAnalytics (PID 2988), HPCommRecovery (PID 4420), and MsMpEng.exe (PID 4808), with hex dumps and disassembly.

Finding: Three processes were flagged: TouchpointAnalytics and HPCommRecovery (both pre-installed HP laptop utilities) and MsMpEng.exe (Windows Defender's own scanning engine).

TECHNICAL NOTE: ASSESSING THE MALFIND FLAGS

malfind is intentionally broad, it flags the memory permission pattern associated with code injection, not injection itself, so a certain rate of false positives on a clean system is expected and must be triaged rather than reported at face value.

MsMpEng.exe is a textbook example: antivirus engines legitimately allocate RWX memory for their own emulation/unpacking sandboxes to safely detonate suspicious code, which is exactly the pattern malfind is designed to catch. Flagging Windows Defender's own process is one of the most common and well-documented malfind false positives.

TouchpointAnalytics and HPCommRecovery are OEM telemetry/support utilities bundled with HP laptops (consistent with the HP hardware identified in the companion Windows Registry examination of this portfolio); vendor utilities of this kind sometimes use RWX regions for legitimate JIT-compiled or self-updating code, another common source of benign flags.

Assessment: all three flags are consistent with expected false positives on a clean baseline system, not evidence of compromise. In a real investigation, the next step for any of these would be to export the flagged region (--dump), hash it, and check it against a known-good hash or submit it for reputation/YARA analysis rather than accept or dismiss the flag on process name alone.

5.10 windows.filescan: Open File Handles

Purpose: Scans memory for file-object structures, recovering the names of files that were open (or recently open) at capture time, including system files, NTFS metadata structures, and driver files, which is useful for identifying files a suspicious process had open that may not otherwise be visible from a live directory listing.


```

C:\Users\Awais\Desktop\CCFA\3. Tools\volatility3-develop>python vol.py -f "C:\Users\Awais\Desktop\CCFA\3. Tools\DumpIt-mirror-main\LAPTOP-QRAGD169-20260722-053441.dmp"
windows.filescan
Volatility 3 Framework 2.28.1
Progress: 100.00 PDB scanning finished
Offset Name
0xbb8d1787c210 \Windows\System32\drivers\npfs.sys
0xbb8d1787c380 \Windows\System32\drivers\netbt.sys
0xbb8d1787c660 \Windows\System32\drivers\tdx.sys
0xbb8d1787cab0 \Directory
0xbb8d1787cc20 \Directory
0xbb8d1787d070 \Windows\System32\drivers\wmfs.sys
0xbb8d1787d1e0 \Windows\System32\DriverStore\FileRepository\basicdisplay.inf_amd64_19e58b6267591a82\BasicDisplay.sys
0xbb8d1787d4c0 \Windows\System32\drivers\cinfs.sys
0xbb8d1787d910 \Windows\System32\DriverStore\FileRepository\basicrender.inf_amd64_eebb5a8467eb88ec\BasicRender.sys
0xbb8d1787d960 \Windows\System32\drivers\tdi.sys
0xbb8d1788f210 \Windows\System32\drivers\pacer.sys
0xbb8d1788f4f0 \Windows\System32\drivers\netbios.sys
0xbb8d1788f7d0 \Windows\System32\drivers\afd.sys
0xbb8d1788f940 \Windows\System32\drivers\ndisapi.sys
0xbb8d1788fc20 \Windows\System32\drivers\afunix.sys
0xbb8d17890350 \Windows\System32\drivers\vid.sys
0xbb8d17890e0 \Windows\System32\drivers\vwifflt.sys
0xbb8d1789380 \ShiftMirr
0xbb8d17894f0 \LogFile
0xbb8d1789660 \MapAttributeValue
0xbb8d17897d0 \Secure:$II:$INDEX_ALLOCATION
0xbb8d1789d90 \Shift
0xbb8d1789a070 \Extend:$I30:$INDEX_ALLOCATION
0xbb8d1789a350 \Secure:$SDS:$DATA

```

Figure 11. windows.filescan output listing open file objects (npfs.sys, netbt.sys, tdx.sys, afd.sys) and NTFS metadata structures (\$LogFile, \$Secure, \$Extend).

Finding: The recovered file handles were entirely standard Windows kernel driver files and NTFS metadata structures (e.g. \$LogFile, \$Secure:\$SDS:\$DATA), consistent with normal system operation and containing no unexpected or user-suspicious file paths.

6. Narrowing an Investigation: Targeted Filtering

The plugins above were run unfiltered, returning every process/connection/region system-wide, appropriate for an initial baseline triage. Once a specific process becomes a focus of interest (for example, one of the malfind hits in Section 5.9, in a case where it could not be dismissed as a false positive), Volatility supports narrowing any plugin's scope directly, rather than manually filtering its full output:

- `--pid`, restrict the plugin to a single specific process ID.
- `--name`, filter by process name, supporting wildcard patterns.
- `--regex`, filter using a full regular expression for more complex pattern matching.
- `--dump`, combined with malfind, exports the flagged memory region itself to disk for hashing, YARA scanning, or manual reverse-engineering, rather than only reporting its presence.

For example, to focus malfind on a single suspicious PID:

```
# Target a specific suspicious PID vol -f memory.dmp windows.malware.malfind --pid 1724
```

To filter by process name pattern instead of a fixed PID:

```
# Filter by process name pattern vol -f memory.dmp windows.malware.malfind --name "powershell*"
```

Combining a targeted PID filter with `--dump` extracts the flagged region itself for offline analysis:

```
# Use regex to find injected code in specific processes vol -f memory.dmp windows.malware.malfind --pid 1724 --dump
```

Beyond these, Volatility also supports narrowing by memory region using `--base`, or combining several of the filters above at once, to isolate only the most relevant artifacts for a given threat model rather than paging through a full system-wide dump on every plugin run.

7. Findings Summary and Conclusion

TECHNICAL NOTE: OVERALL ASSESSMENT

System identification (windows.info) confirmed a valid Windows 10 image with a correctly resolved symbol table.

Process listing and hierarchy (windows.pslist, windows.pstree) showed an expected, normally-parented Windows process set with no orphaned or misplaced processes.

Network connections (windows.netscan) resolved entirely to legitimate, identifiable applications with no unrecognized foreign endpoints.

Command-line arguments (windows.cmdline) showed every svchost.exe instance carrying a valid, named service-group argument.

Injected-code detection (windows.malware.malfind) flagged three processes; all three were assessed as expected false positives (an antivirus engine and two OEM utilities) rather than evidence of compromise.

Open file handles (windows.filescan) showed only standard system driver and NTFS metadata files.

This exercise reproduces the standard first-pass triage workflow an examiner runs against any newly acquired Windows memory image: confirm the image and profile, enumerate processes and their relationships, check network activity, and screen for injected code, before deciding whether deeper analysis is warranted. In this case, every plugin's output was consistent with a normal, uncompromised baseline system, but the same seven-plugin sequence, together with the targeted --pid/--name/--regex/--dump filtering shown in Section 6, is exactly what would be applied first against a genuinely suspicious system, where the findings at each step would be expected to look materially different.